

## ✓ Task 4 — Responsible AI & Model Interpretation

### Project: Spam Email Detection using Machine Learning

Objective: To analyze model feature importance, explain individual predictions using LIME, check for possible bias, and propose practical mitigation steps.

```
# Import required libraries
import pandas as pd
import numpy as np

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Import machine learning libraries

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)

print("Machine learning libraries imported successfully!")
```

Machine learning libraries imported successfully!

```
# Download the spam email dataset

import urllib.request

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/spam_email_dataset/spam_email_dataset.tsv"

urllib.request.urlretrieve(url, "spam_dataset.tsv")

print("Dataset downloaded successfully!")
```

Dataset downloaded successfully!

```
# Load the dataset
```

```
df = pd.read_csv(
    "spam_dataset.tsv",
    sep="\t",
    header=None,
    names=["label", "message"]
)

print("Dataset loaded successfully!")
print("Number of rows:", len(df))
print("\nFirst 5 rows:")
display(df.head())
```

Dataset loaded successfully!  
Number of rows: 5572

First 5 rows:

|   | label | message                                           |
|---|-------|---------------------------------------------------|
| 0 | ham   | Go until jurong point, crazy.. Available only ... |
| 1 | ham   | Ok lar... Joking wif u oni...                     |
| 2 | spam  | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham   | U dun say so early hor... U c already then say... |
| 4 | ham   | Nah I don't think he goes to usf, he lives aro... |

```
# Check for missing values and duplicate messages

print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates().reset_index(drop=True)

print("\nRows after removing duplicates:", len(df))
```

Missing values:  
label 0  
message 0  
dtype: int64

Duplicate rows: 403

Rows after removing duplicates: 5169

```
# Step 7 – Train/Test Split

from sklearn.model_selection import train_test_split

# Separate features and target
X = df["message"]
y = df["label"].map({"ham": 0, "spam": 1})

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 4135
Testing samples: 1034
```

```
# Step 8 – TF-IDF Feature Extraction

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words="english",
    max_features=5000
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("TF-IDF transformation completed!")
print("Training feature shape:", X_train_tfidf.shape)
print("Testing feature shape:", X_test_tfidf.shape)
```

```
TF-IDF transformation completed!
Training feature shape: (4135, 5000)
Testing feature shape: (1034, 5000)
```

```
# Step 9 – Train Logistic Regression

from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train_tfidf, y_train)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

#### # Step 10 – Model Performance

```
from sklearn.metrics import accuracy_score, precision_score

y_pred = lr_model.predict(X_test_tfidf)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
```

```
Accuracy : 0.9584139264990329
Precision: 0.9782608695652174
```

#### # Step 11 – Feature Importance

```
feature_names = tfidf.get_feature_names_out()
coefficients = lr_model.coef_[0]

feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": coefficients
})

# Top features indicating Spam
top_spam = feature_importance.sort_values(
    by="Importance",
    ascending=False
).head(15)

print("Top features associated with Spam:")
display(top_spam)
```

Top features associated with Spam:

|      | Feature | Importance |
|------|---------|------------|
| 4701 | txt     | 4.181886   |
| 3279 | mobile  | 3.695900   |
| 4943 | www     | 3.454981   |
| 4715 | uk      | 3.334554   |
| 708  | claim   | 3.231580   |
| 4116 | reply   | 3.196726   |
| 1761 | free    | 3.136740   |
| 4579 | text    | 3.084441   |
| 4453 | stop    | 3.072474   |
| 90   | 150p    | 2.677542   |
| 3990 | prize   | 2.645560   |
| 182  | 50      | 2.613178   |
| 4261 | service | 2.577186   |
| 3472 | new     | 2.552524   |
| 754  | com     | 2.465700   |

```
import matplotlib.pyplot as plt

# Step 12 – Plot Top Spam Features

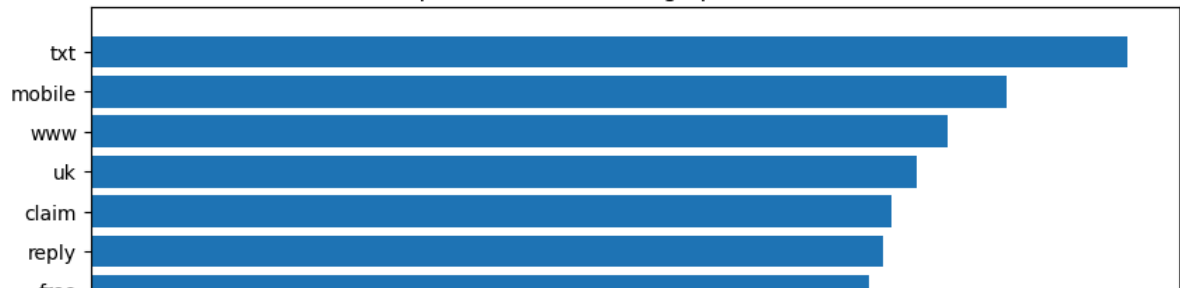
plt.figure(figsize=(10, 6))

plt.barh(
    top_spam["Feature"],
    top_spam["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top Features Influencing Spam Prediction")
plt.gca().invert_yaxis()

plt.show()
```

Top Features Influencing Spam Prediction



# Step 13 – Install LIME

```
!pip install lime
```

Collecting lime

Downloading lime-0.2.0.1.tar.gz (275 kB)

275.7/275.7 kB 6.9 MB

Preparing metadata (setup.py) ... done

Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/di  
 Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-pa  
 Requirement already satisfied: scipy in /usr/local/lib/python3.13/dist-pa  
 Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-pac  
 Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/pytho  
 Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/pytho  
 Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.13  
 Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.13/  
 Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/p  
 Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/pyth  
 Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.13  
 Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3  
 Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13  
 Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/pyt  
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3  
 Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.13/  
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python  
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python  
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3  
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/pyt  
 Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3  
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist

Building wheels for collected packages: lime

Building wheel for lime (setup.py) ... done

Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283

Stored in directory: /root/.cache/pip/wheels/7c/04/5c/157dc9106512a6c7a

Successfully built lime

Installing collected packages: lime

Successfully installed lime-0.2.0.1

# Step 14 – LIME Explanation

```
from lime.lime_text import LimeTextExplainer
```

```
explainer = LimeTextExplainer(  
    class_names=["Ham", "Spam"]
```

```
)  
  
print("LIME explainer created successfully!")
```

LIME explainer created successfully!

```
# Prediction function for LIME  
  
def predict_proba(texts):  
    texts_tfidf = tfidf.transform(texts)  
    return lr_model.predict_proba(texts_tfidf)
```

```
# Step 16 – Select a test message  
  
sample_message = X_test.iloc[0]  
  
print("Test message:")  
print(sample_message)  
  
prediction = lr_model.predict(  
    tfidf.transform([sample_message])  
)[0]  
  
if prediction == 1:  
    print("\nModel Prediction: SPAM")  
else:  
    print("\nModel Prediction: HAM")
```

Test message:  
Good morning, my Love ... I go to sleep now and wish you a great day full  
  
Model Prediction: HAM

```
# Step 17 – Generate LIME explanation  
  
explanation = explainer.explain_instance(  
    sample_message,  
    predict_proba,  
    num_features=10  
)  
  
print("LIME explanation:")  
for word, weight in explanation.as_list():  
    print(word, ":", weight)
```

LIME explanation:  
Good : -0.005338943577901647  
day : -0.0042276783209422375  
thought : -0.004153836411449714  
morning : -0.0034345682848689776  
Love : -0.0033279983032339575  
feeling : 0.003289439491153962  
LOVE : -0.003240361403723202  
great : -0.0027573664288005934

```
sleep : -0.002677710645719782
opportunity : 0.0025007960191582245
```

```
# Step 18 – Display LIME explanation

from IPython.display import display, HTML

html = explanation.as_html()
display(HTML(html))
```

### Prediction probabilities

|      |                        |      |
|------|------------------------|------|
| Ham  | <div><div></div></div> | 0.98 |
| Spam | <div><div></div></div> | 0.02 |

Ham

Spam

|             |      |
|-------------|------|
| Good        | 0.01 |
| day         | 0.00 |
| thought     | 0.00 |
| morning     | 0.00 |
| Love        | 0.00 |
| feeling     | 0.00 |
| LOVE        | 0.00 |
| great       | 0.00 |
| sleep       | 0.00 |
| opportunity | 0.00 |

### Text with highlighted words

Good morning, my Love ... I go to sleep now and wish you a great day full of feeling better and opportunity ... You are my last thought babe, I LOVE YOU \*kiss\*

```
# Step 19 – Bias / Class Distribution Check

class_counts = df["label"].value_counts()

print("Class Distribution:")
print(class_counts)

# Calculate percentages
class_percentages = df["label"].value_counts(normalize=True) * 100
```

```
Class Distribution:
label
ham      4516
spam     653
Name: count, dtype: int64

Class Percentages:
label
ham      87.37
spam     12.63
Name: proportion, dtype: float64
```

## Step 20 — Mitigation Recommendations

The following practical steps can be used to improve the reliability and fairness of the spam detection model:

1. Use a balanced dataset containing enough ham and spam messages.
2. Remove duplicate, incomplete, and poor-quality messages from the dataset.
3. Regularly update the dataset with new types of spam messages.
4. Monitor false positives because legitimate emails should not be incorrectly classified as spam.
5. Monitor false negatives because some spam emails may be incorrectly classified as ham.
6. Use LIME or SHAP explanations to understand important features affecting predictions.
7. If future datasets contain sensitive group attributes, perform group-wise fairness analysis.
8. Retrain and evaluate the model regularly when new email patterns appear.

## Step 21 — Conclusion

In this task, the Spam Email Detection model was analyzed from a Responsible AI perspective. Feature importance was used to identify the words that strongly influenced spam predictions. while LIME was used to explain individual model predictions.